

5/8/2026

Test or Debug Source Code to Ensure Client - SDT6211

Formative Assessment 2

Willem Gerber

1.1 Testing Objectives

System Purpose:

The Umoja Learning Management System is a cloud-enabled academic administration system designed to support the following core activities:

- student registration
- login authentication
- course enrolment
- marks capture
- report generation

Testing Goals:

As the appointed Junior Software Tester, my goal is to test the Umoja Learning Management System application in a structured and professional manner to confirm whether the system is ready for deployment.

The testing process will aim to:

- verify that each feature works according to the client requirements
- identify defects that could affect accuracy, security, usability, or performance
- confirm that incorrect login behaviour is detected and documented
- verify that student average calculations are correct and consistent
- check that course enrolment follows the correct business rules
- assess report generation speed and responsiveness
- ensure that meaningful error messages are displayed when invalid input is entered
- provide clear evidence for debugging, reporting, and retesting

Scope of Testing

Included in scope:

- login authentication testing
- student registration testing
- course enrolment testing
- marks capture and average calculation testing
- report generation testing
- input validation and error handling testing
- navigation testing between forms

Excluded from this task:

- browser compatibility testing for a web version, because the current ULMS build is a Windows Forms desktop application
- live production deployment testing
- server or network infrastructure testing
- full database stress testing

Main Problem Areas to Be Tested

Based on the project brief and inspection of the current build, the following areas require special attention during testing:

- incorrect login validation behaviour
- inconsistent calculation of student averages
- incorrect course enrolment rules
- slow performance when loading reports
- missing error handling messages
- incomplete testing documentation

1.2 Testing Types Identification

Testing Type	Purpose in ULMS	Example Test Focus
Functional Testing	Verify each feature behaves according to the stated requirements.	Confirm that login only succeeds with the correct username and password combination.
Input Validation Testing	Ensure fields reject invalid, missing, or incorrectly formatted values.	Enter letters in marks fields, blank student IDs, negative age values, or empty course selections.
Integration Testing	Verify that different modules work together within the intended user workflow.	Log in, open dashboard, register a student, and enroll that student into a course.
Performance Testing	Measure whether key screens and outputs respond within acceptable time limits.	Time how long it takes to generate Student Summary, Marks, and Enrollment reports.
Usability Testing	Confirm navigation, labels, and messages are clear and user-friendly.	Check whether users can move from dashboard to forms and back without confusion.
Security Testing	Verify that authentication and access controls prevent unauthorized access.	Attempt login using incorrect credentials, partially correct credentials, and blank inputs.

Functional Testing

Functional testing will be performed to confirm that each ULMS feature behaves as expected.

Areas to test:

- successful login with valid credentials
- rejection of login with invalid credentials
- successful student registration when all required details are entered
- successful course enrolment when valid student and course details are selected
- correct calculation of marks average and final result
- correct generation of student, marks, and enrolment reports

Input Validation Testing

Input validation testing will ensure that data entered the system is checked before processing.

Examples of validation checks:

- blank username and password fields
- empty student registration fields
- invalid email format
- non-numeric age or marks values
- missing course or semester selection during enrolment
- selecting Generate Report without choosing a report type

Integration checks:

- login opens the dashboard successfully
- dashboard buttons open the correct forms
- closing or returning from forms does not crash the application
- a student can be registered and then processed through enrolment and marks-related activities in the expected workflow

Performance Testing

Performance testing will focus mainly on report generation because the brief identifies slow loading as a problem.

Performance checks will include:

- measuring the time taken to load each report type
- checking whether the system becomes unresponsive while reports are loading
- comparing actual load time against an acceptable response target for desktop use

Usability Testing

Usability testing will evaluate the user experience of the application.

Areas to review:

- clarity of labels, buttons, and output messages
- consistency of Save, Clear, Back, Login, and Generate buttons
- ease of moving between screens
- readability of displayed results and reports

Security Testing

Security testing will focus on authentication and user access.

Areas to test:

- correct username with incorrect password
- incorrect username with correct password
- blank credentials
- repeated invalid login attempts
- access to dashboard functions without proper login

1.3 Testing Environment Setup

Hardware

The hardware environment for testing will include:

- desktop or laptop computer
- Intel Core i5 processor or equivalent
- minimum 8 GB RAM
- at least 10 GB available storage
- standard keyboard, mouse, and monitor

Software

The software environment for testing will include:

- Windows 10 or Windows 11 operating system
- ULMS application build
- .NET SDK for compiling and running the project
- Visual Studio or Visual Studio Code for debugging and code tracing
- SQL Server for future data testing where database support is required
- Microsoft Word for compiling formal documentation and evidence

Browser / OS

Because the current ULMS version is a Windows Forms desktop application, browser testing is not required for the present build. The main operating systems for testing will be:

- Windows 10
- Windows 11

Example Tools Used

The following tools and resources will support testing activities:

- Visual Studio or Visual Studio Code
- SQL Server
- .NET CLI tools
- stopwatch or timer for performance measurement
- screenshots for defect evidence
- prepared test data sets containing valid, invalid, and boundary values

Test Data Sets

The following test data will be prepared for use during testing:

- valid login credentials
- invalid login credentials
- valid student registration details
- missing and incorrect student details
- valid course enrolment selections
- incomplete course enrolment data
- marks values at boundary levels such as 0, 49, 50, and 100
- report requests for each available report type

Task 2: Test Case Design

Test Case Design Overview

The following test cases have been designed to verify both normal and abnormal behaviour in the ULMS application. Each test case includes a unique test case ID, a short description, the test input, the expected output, the actual output field for execution evidence, and the test status.

For this design stage:

- Actual Output is recorded as ` Pending Execution `
- Status is recorded as ` Not Executed `

Test Case ID	Description	Input	Expected Output	Actual Output	Status
LOGIN-001	Verify that the system allows login with valid credentials.	Username: adminPassword: 1234	User should be logged in successfully and the dashboard should open.	Pending Execution	Not Execute
LOGIN-002	Verify that the system rejects login with invalid credentials	Username: user1Password: 9999	System should display an invalid login message and remain on the login form.	Pending Execution	Not Executed
LOGIN-003	Verify that the system rejects login when the username is correct but the password is incorrect.	Username: admin Password: 9999	System should deny access and display an invalid login message.	Pending Execution	Not Executed
LOGIN-004	Verify that the system rejects login when the username is incorrect but the password is correct.	Username: guest Password: 1234	System should deny access and display an invalid login message.	Pending Execution	Not Executed
ENROLL-001	Verify that a student can be enrolled successfully with valid details.	Student ID: ST001 Student Name: Test Student Course: Programming 1 Semester: Semester 1	System should complete enrolment and display the correct student and course details.	Pending Execution	Not Executed

ENROLL-002	Verify that duplicate course enrolment is prevented for the same student and semester.	Student ID: ST001 Student Name: Test Student Course: Programming 1 Semester: Semester 1 entered twice	System should reject the second enrolment attempt and display a duplicate enrolment warning.	Pending Execution	Not Executed
ENROLL-003	Verify that course enrolment does not proceed when required fields are missing.	Student ID: ST001 Student Name: Course: Semester: Semester 1	System should not complete enrolment and should display a validation error message.	Pending Execution	Not Executed
MARKS-001	Verify that marks capture accepts valid numeric values.	Student ID: ST001 Student Name: Test Student Subject1: 70 Subject2: 80 Subject3: 90	System should process the marks, calculate the average correctly, and display a final result.	Pending Execution	Not Executed
MARKS-002	Verify that marks capture rejects non-numeric values.	Student ID: ST001 Student Name: Test Student Subject1: A Subject2: 70 Subject3: 80	System should not process the marks and should display a numeric validation error message.	Pending Execution	Not Executed
MARKS-003	Verify that marks capture rejects blank subject fields.	Student ID: ST001 Student Name: Test Student Subject1: `` Subject2: 60 Subject3: 75	System should not process the marks and should request all required values.	Pending Execution	Not Executed
AVG-001	Verify average calculation at the fail boundary.	Subject1: 40 Subject2: 50 Subject3: 59	Average should be calculated as 49.67 and the final result should be Fail	Pending Execution	Not Executed
AVG-002	Verify average calculation at the pass boundary.	Subject1: 50 Subject2: 50 Subject3: 50	Average should be calculated as 50 and the final result should be Pass	Pending Execution	Not Executed
AVG-003	Verify that high marks produce the correct average and pass result.	Subject1: 100 Subject2: 90 Subject3: 80	Average should be calculated as 90 and the final result should be Pass	Pending Execution	Not Executed
REPORT-001	Verify that the Student Summary Report generates the correct output.	Report Type: Student Summary Report Student ID: ST001	System should generate a report containing the correct report title, selected student ID, and student summary details.	Pending Execution	Not Executed

TASK 3

FUNCTIONAL TEST EXECUTION

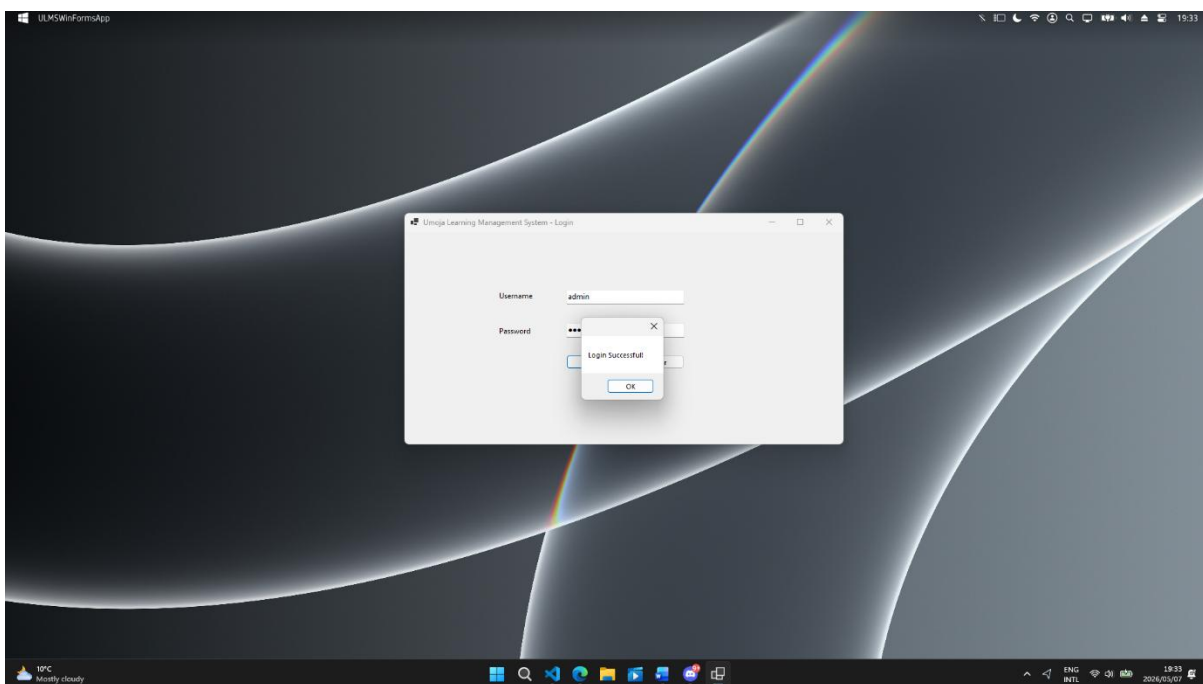
LOGIN-001 Valid Login Credentials

Field	Detail
Test Case ID	LOGIN-001
Description	Verify that the system allows login with valid credentials
Input	Username: admin / Password: 1234
Expected Output	Login Successful message is displayed and the Dashboard form opens
Actual Output	Message box displayed "Login Successful!" and the Dashboard form opened
Status	Pass

Execution Log:

Entered admin as the username and 1234 as the password and clicked the Login button. The system displayed the Login Successful message box and opened the dashboard as expected.

Screenshot Reference:



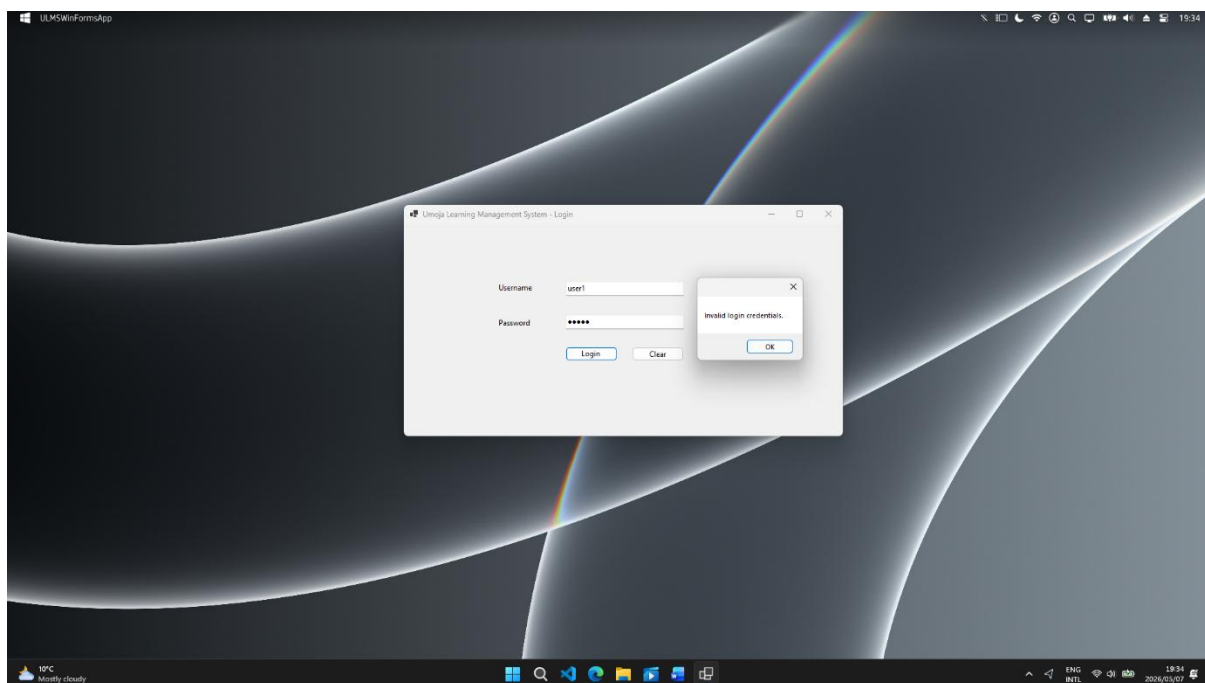
LOGIN-002 Fully Invalid Credentials

Field	Detail
Test Case ID	LOGIN-002
Description	Verify that the system rejects login when both username and password are invalid.
Input	Username: user1 / Password: 9999
Expected Output	System displays an invalid login message and remains on the login form
Actual Output	Message box displayed "Invalid login credentials." and the login form remained open.
Status	Pass

Execution Log:

Entered user1 as the username and 9999 as the password and clicked the Login button. Neither value matched the accepted username or password, so the system correctly rejected the attempt.

Screenshot Reference:



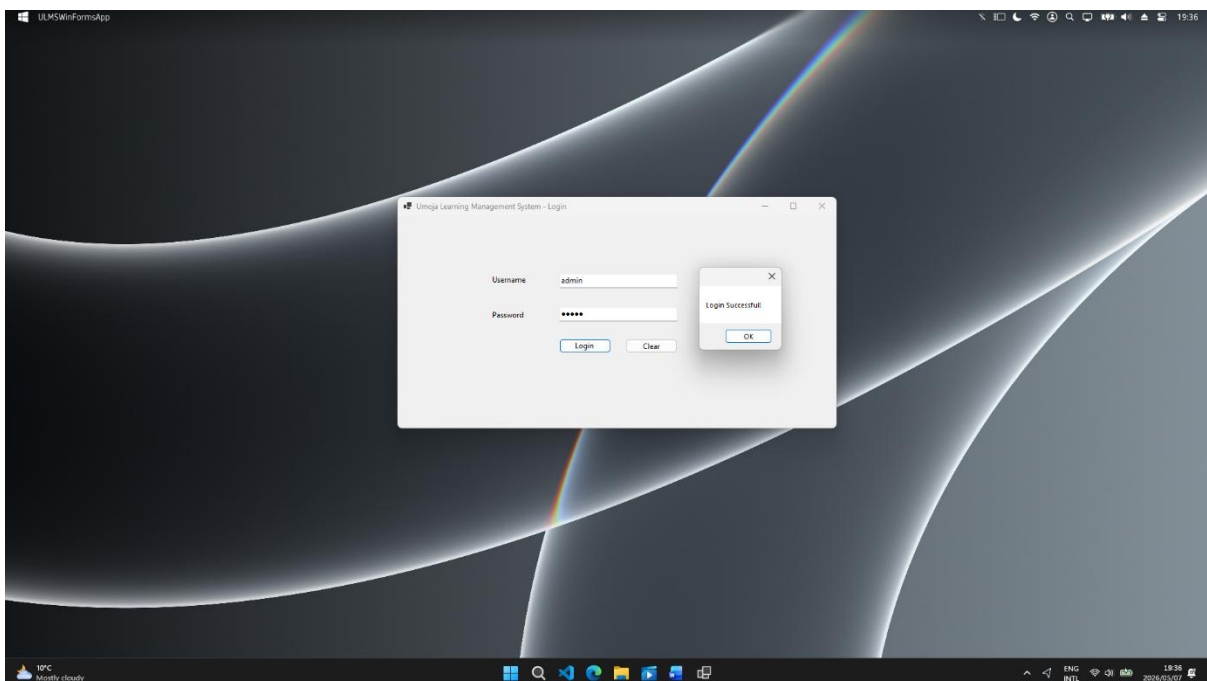
LOGIN-003 Correct Username, Incorrect Password

Field	Detail
Test Case ID	LOGIN-003
Description	Verify that the system rejects login when the username is correct but the password is incorrect
Input	Username: admin / Password: 9999
Expected Output	System denies access and displays an invalid login message
Actual Output	Message box displayed "Login Successful!" and the Dashboard form opened. Access was granted incorrectly
Status	Fail

Execution Log:

Entered admin as the username and 9999 as the password. The login condition uses a logical OR operator (||) instead of AND (&&), meaning the system grants access if either the username OR the password is correct. Because the username matched, access was granted even though the password was wrong

Screenshot Reference:



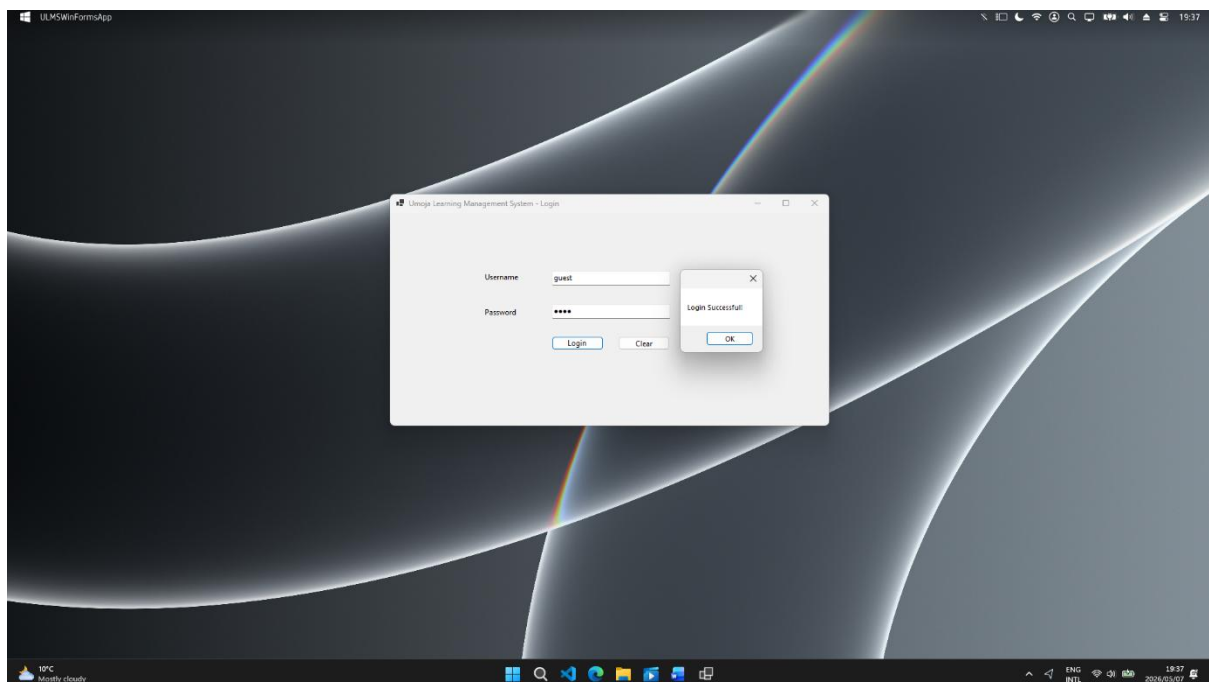
LOGIN-004 Incorrect Username, Correct Password

Field	Detail
Test Case ID	LOGIN-004
Description	Verify that the system rejects login when the username is incorrect but the password is correct
Input	Username: guest / Password: 1234
Expected Output	System denies access and displays an invalid login message.
Actual Output	Message box displayed "Login Successful!" and the Dashboard form opened. Access was granted incorrectly
Status	Fail

Execution Log:

Entered guest as the username and 1234 as the password. The OR operator bug caused the system to grant access because the password matched, even though the username was not recognised. This confirms the same defect identified in TC-LOGIN-003.

Screenshot Reference:



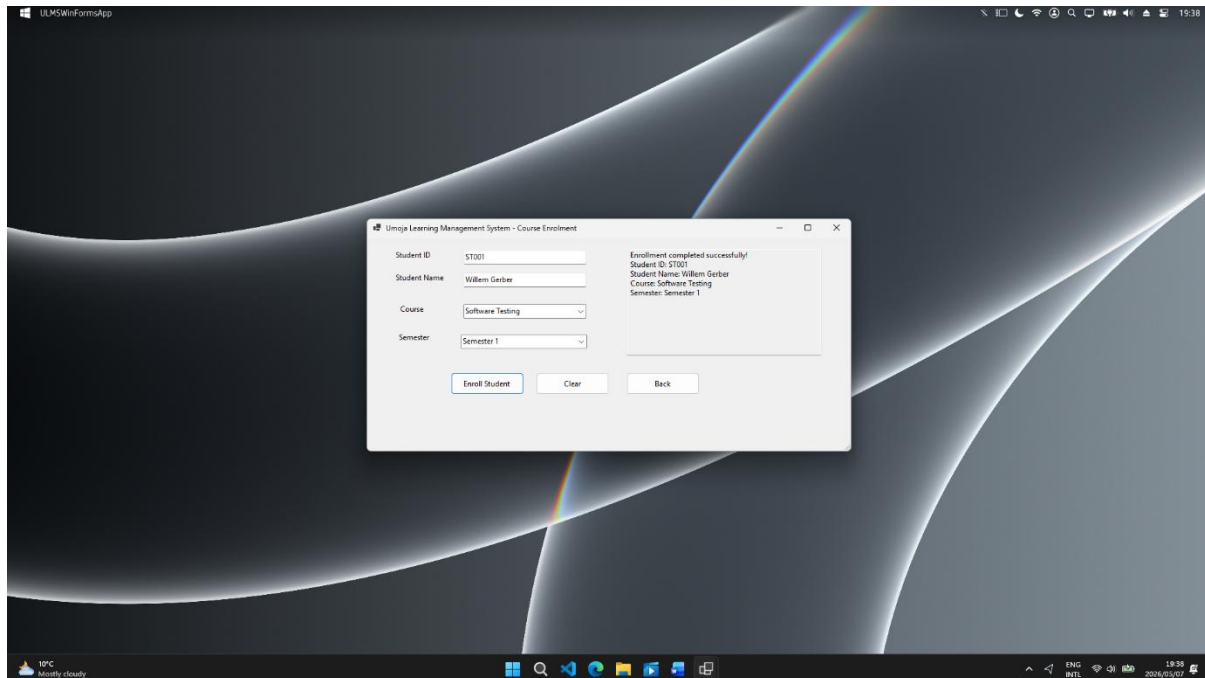
ENROLL-001 Valid Enrolment

Field	Detail
Test Case ID	ENROLL-001
Description	Verify that a student can be enrolled successfully with valid details
Input	Student ID: ST001 / Student Name: Willem Gerber / Course: Programming 1 / Semester: Semester 1
Expected Output	System completes enrolment and displays student and course details in the output area
Actual Output	Enrolment output displayed all entered details correctly. No errors occurred
Status	Pass

Execution Log:

Entered all required enrolment details and clicked the Enrol button. The output area displayed the student ID, name, selected course, and semester as entered

Screenshot Reference:



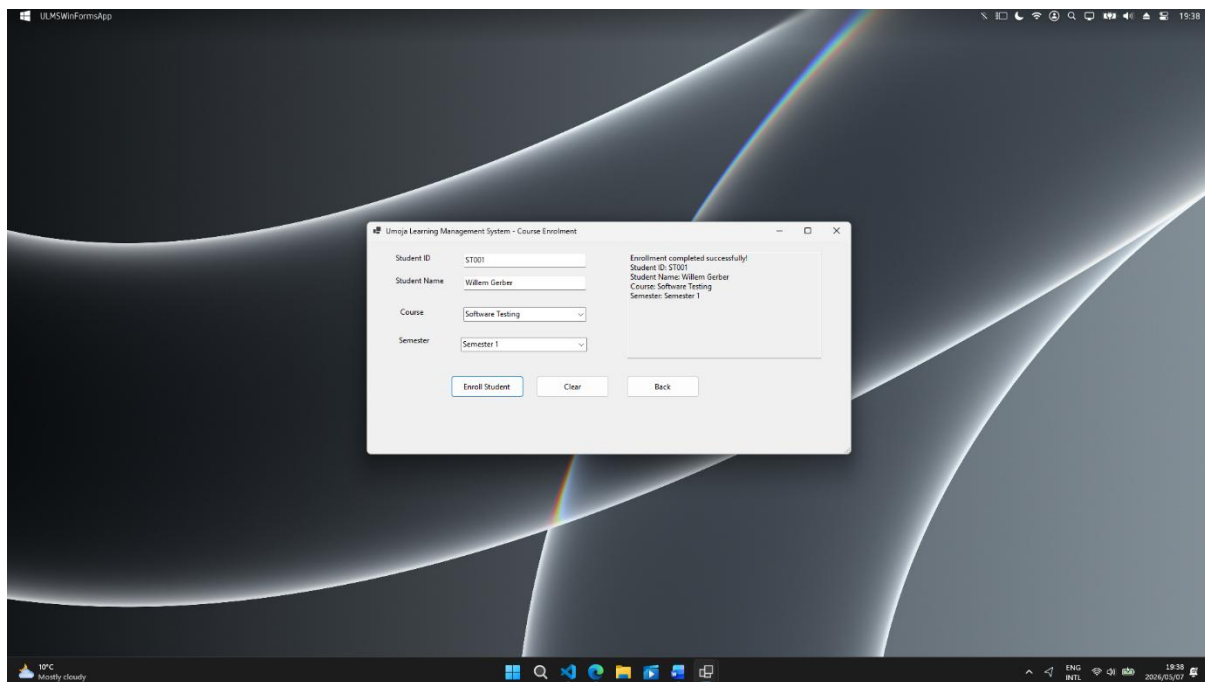
ENROLL-002 Duplicate Enrolment

Field	Detail
Test Case ID	ENROLL-002
Description	Verify that duplicate course enrolment is prevented for the same student and semester
Input	Same details as TC-ENROLL-001 entered a second time
Expected Output	System should reject the second enrolment attempt and display a duplicate enrolment warning
Actual Output	System accepted the second enrolment without any warning or error. The output was overwritten with identical details and no duplicate check was performed
Status	Fail

Execution Log:

Clicked the Enrol button a second time with the same student and course details already on the form. The system processed the enrolment again and displayed a new success output. No duplicate validation logic exists in the current build

Screenshot Reference:



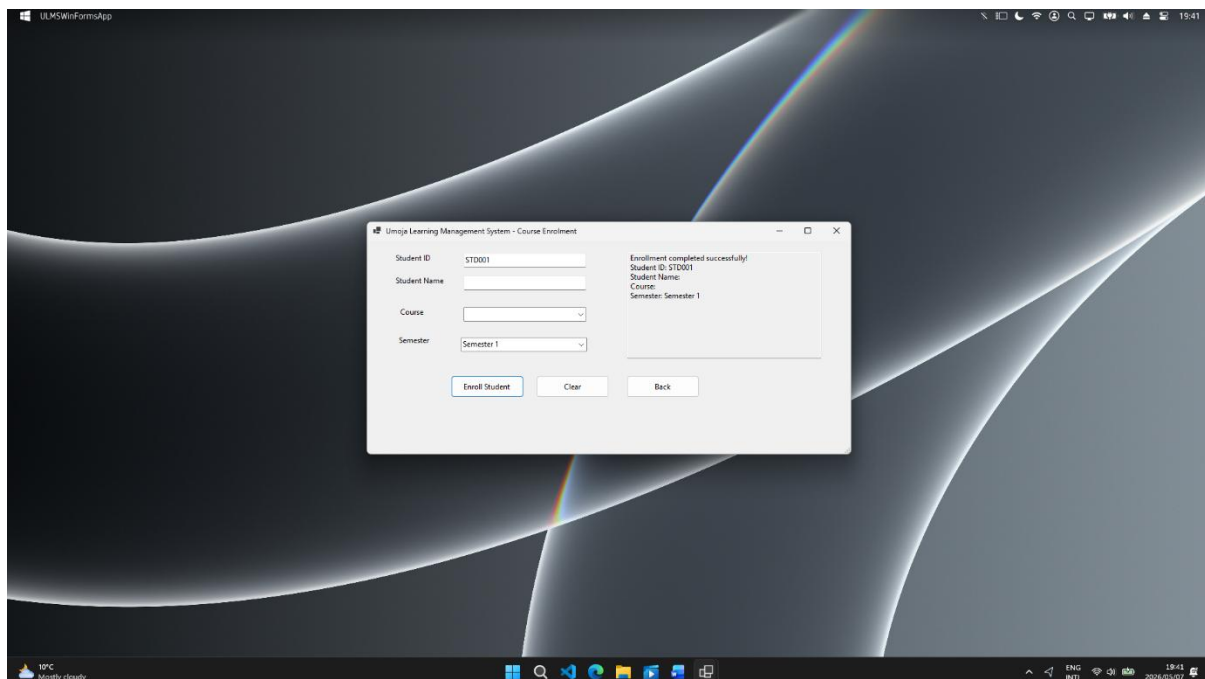
ENROLL-003 Missing Required Fields

Field	Detail
Test Case ID	ENROLL-003
Description	Verify that enrolment does not proceed when required fields are missing
Input	Student ID: `ST001` Student Name: left blank Course: left blank Semester: `Semester 1`
Expected Output	System should not complete enrolment and should display a validation error message
Actual Output	System completed enrolment with blank student name and course values. No validation error was displayed
Status	Fail

Execution Log:

Cleared the Student Name and Course fields and clicked the Enrol button. The system accepted the input without checking whether required fields were populated. The output displayed empty values for student name and course with no error message shown

Screenshot Reference:



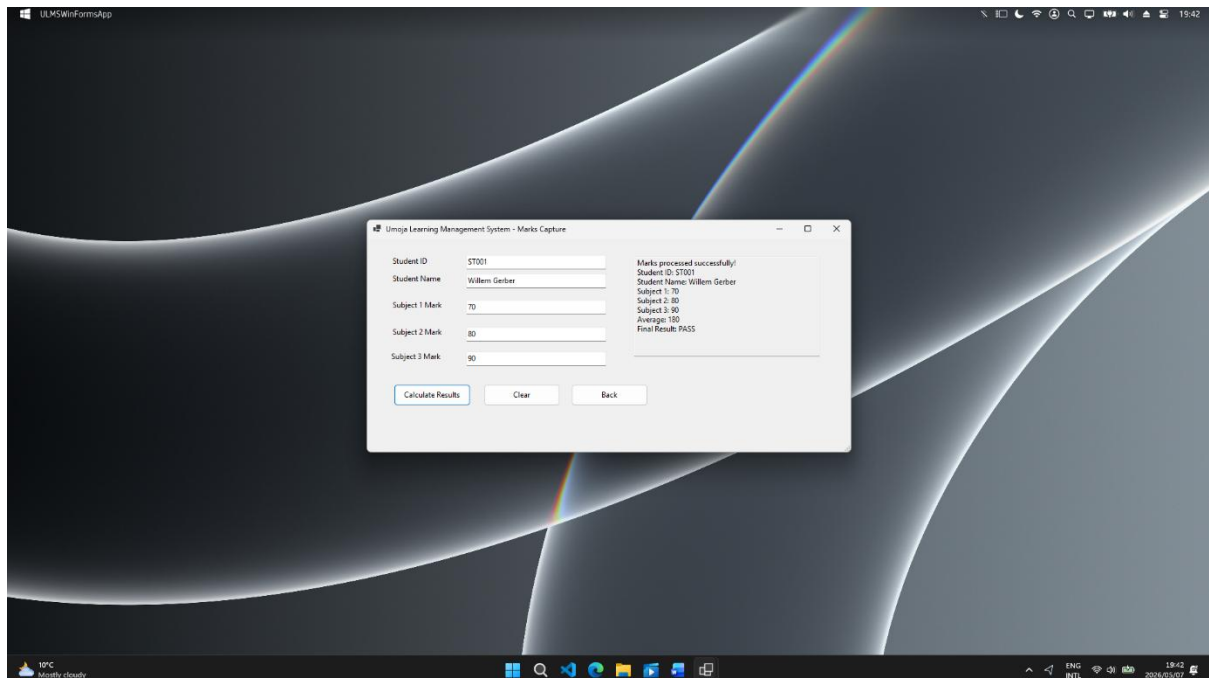
MARKS-001 Valid Marks Entry

Field	Detail
Test Case ID	MARKS-001
Description	Verify that marks capture accepts valid numeric values and calculates correctly
Input	Student ID: `ST001` Student Name: `Willem Gerber` Subject 1: `70` Subject 2: `80` Subject 3: `90`
Expected Output	Average should be `80` and the result should be `PASS`
Actual Output	Average displayed as `180` and result displayed as `PASS`
Status	Fail

Execution Log:

Entered valid numeric marks and clicked Calculate Results. The calculation applied is `Subject1 + Subject2 + Subject3 / 3`, which resolves to `70 + 80 + 30 = 180` due to incorrect operator precedence. The correct calculation should be `(70 + 80 + 90) / 3 = 80`. Although the result status shows PASS, the average is wrong

Screenshot Reference:



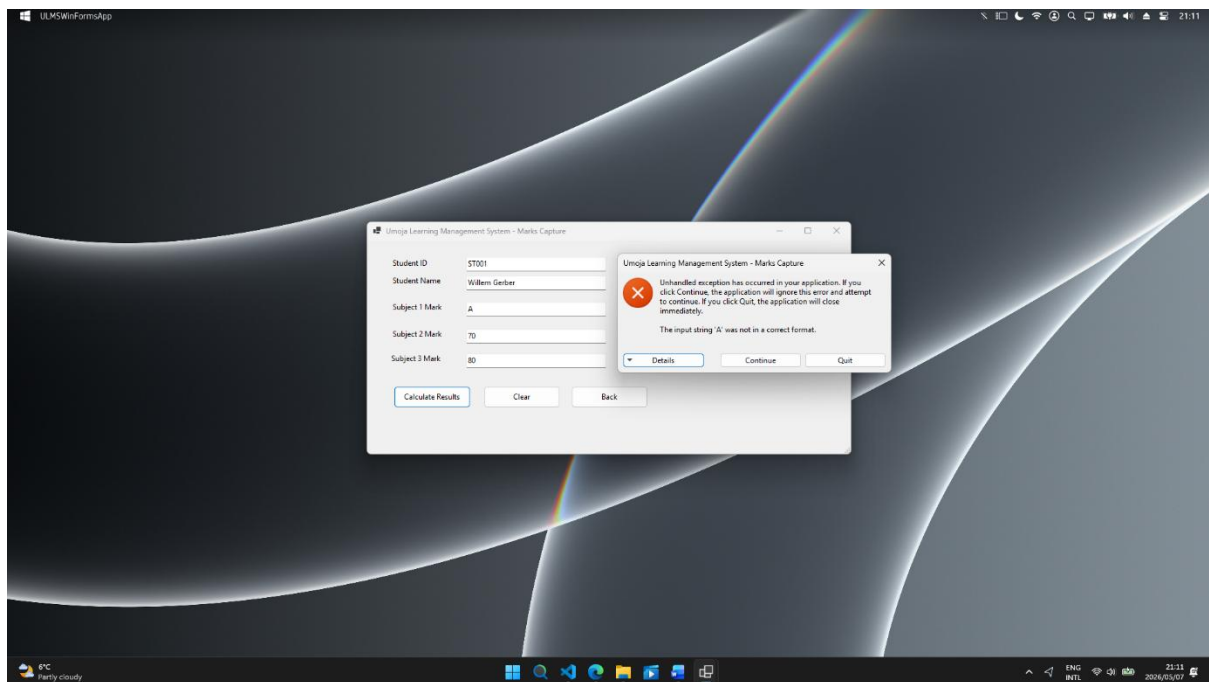
Mark-002 Non-Numeric Marks Entry

Field	Detail
Test Case ID	Marks-002
Description	Verify that marks capture rejects non-numeric values
Input	Student ID: `ST001` Student Name: `Willem Gerber` Subject 1: `A` Subject 2: `70` Subject 3: `80`
Expected Output	System should display a numeric validation error message and not process the marks
Actual Output	Application threw an unhandled FormatException error and crashed when attempting to convert `A` to a number using `Convert.ToDouble`
Status	Fail

Execution Log:

Entered the letter `A` in the Subject 1 field and clicked Calculate Results. The `Convert.ToDouble` call on a non-numeric string caused an unhandled exception. No user-friendly validation message was shown

Screenshot Reference:



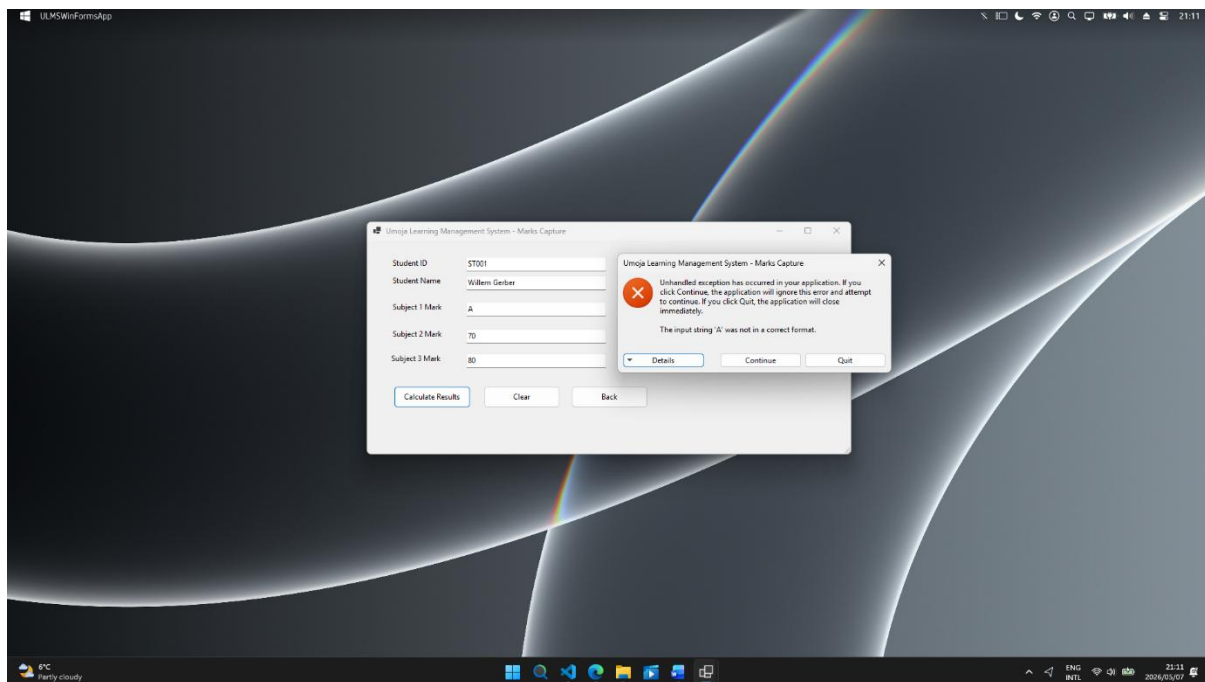
Mark-003 Blank Marks Field

Field	Detail
Test Case ID	MARKS-003
Description	Verify that marks capture rejects blank subject fields
Input	Student ID: `ST001` / Student Name: `Willem Gerber` / Subject 1: left blank / Subject 2: `60` / Subject 3: `75`
Expected Output	System should not process the marks and should prompt the user to complete all required fields
Actual Output	Application threw an unhandled FormatException error when trying to convert an empty string to a number
Status	Fail

Execution Log:

Left the Subject 1 field blank and clicked Calculate Results. An unhandled exception was raised because `Convert.ToDouble("")` fails when no value is provided. No error handling or validation message was present.

Screenshot Reference:



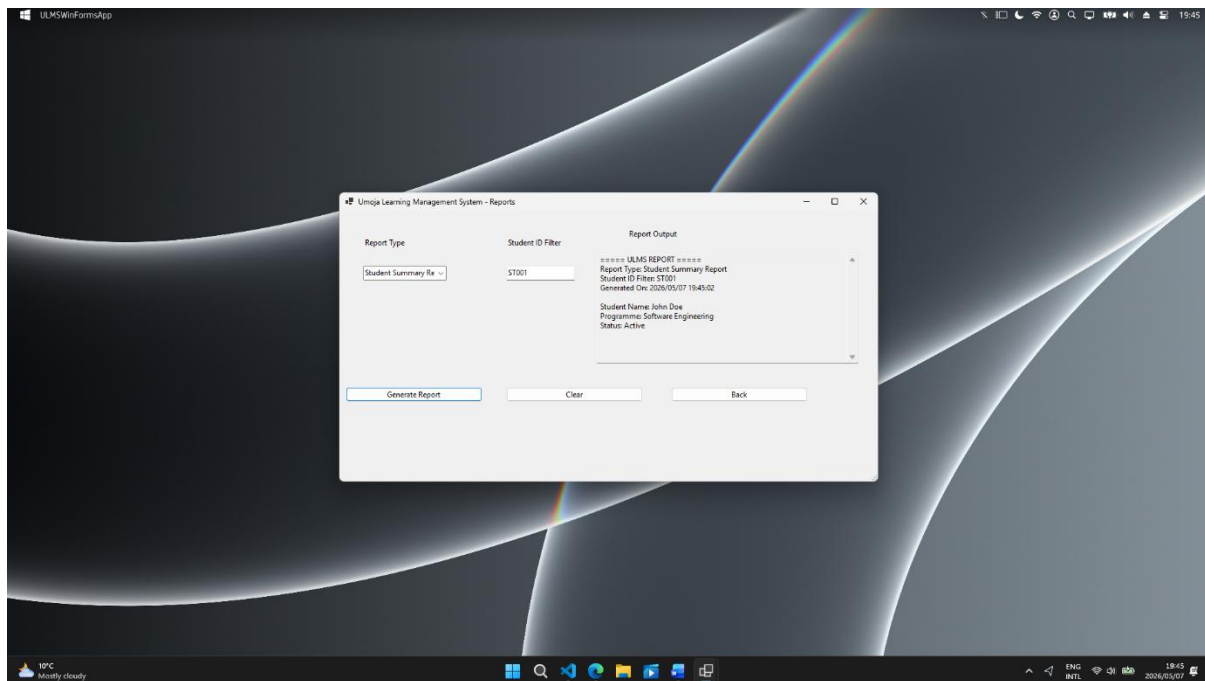
REPORT-001 Student Summary Report

Field	Detail
Test Case ID	REPORT-001
Description	Verify that the Student Summary Report generates correct output
Input	Report Type: ` Student Summary Report ` / Student ID: ` ST001 `
Expected Output	Report displays the correct report title, selected student ID, and student summary details within an acceptable response time
Actual Output	Report generated successfully with the correct title and student summary content. However, the application was visibly unresponsive for approximately 4 seconds before the report appeared
Status	Pass

Execution

Log: Selected Student Summary Report from the dropdown, entered `ST001` as the student ID, and clicked Generate Report. The report appeared after a 4-second delay caused by an intentional `Thread.Sleep(4000)` in the code. Content was correct but the system froze during loading

Screenshot Reference:



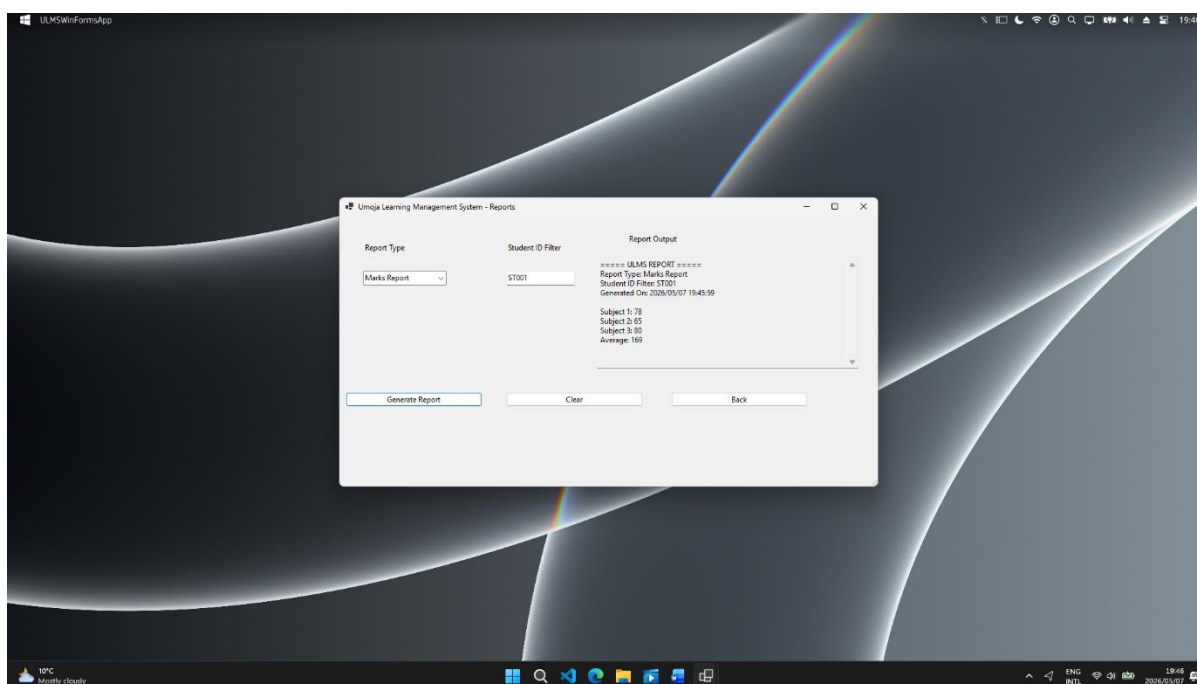
REPORT-002 Marks Report Average Value

Field	Detail
Test Case ID	REPORT-002
Description	Verify that the Marks Report generates correct marks and an accurate average
Input	Report Type: ` Marks Report ` / Student ID: ` ST001 `
Expected Output	Report displays subject marks and the correct calculated average
Actual Output	Report displayed Subject 1: 78, Subject 2: 65, Subject 3: 80, and Average: 169. The average of 169 is incorrect. The correct average for those values is 74.33
Status	Fail

Execution Log:

Selected Marks Report and clicked Generate Report. The report output showed a hardcoded average of 169, which appears to be the incorrect sum of the three subject marks rather than the correct average. This is a data accuracy defect in the report generation logic.

Screenshot Reference:



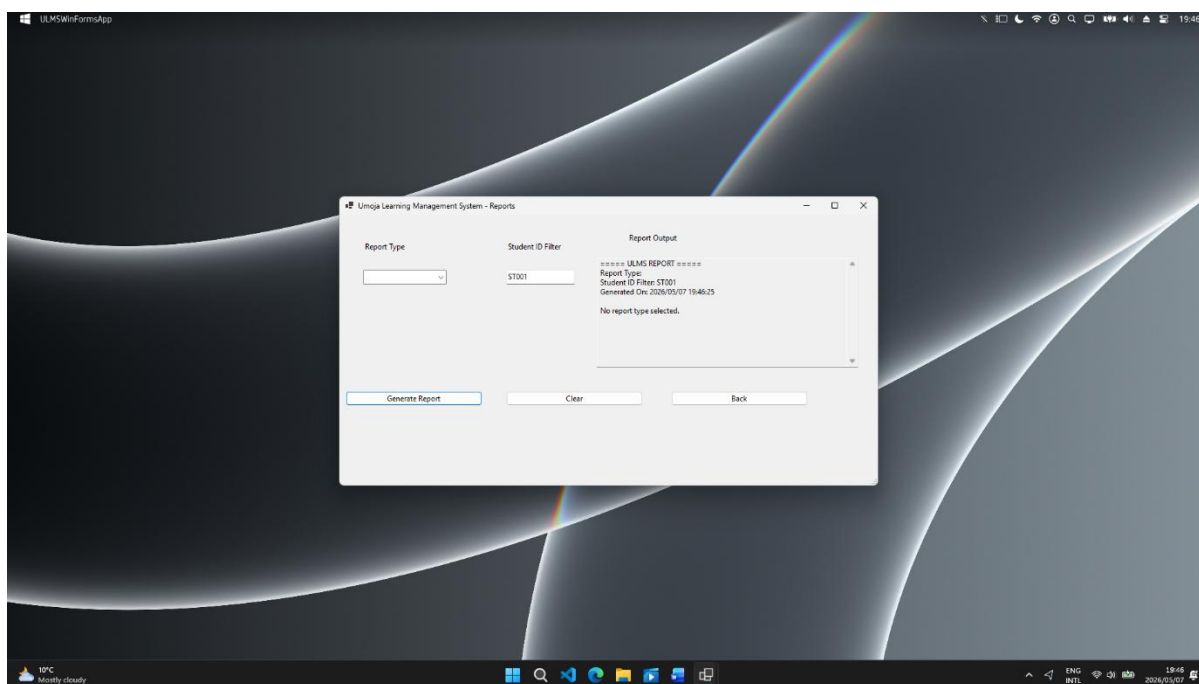
REPORT-003 — Missing Report Type

Field	Detail
Test Case ID	REPORT-003
Description	Verify that report generation handles a missing report type correct
Input	Report Type: not selected / Student ID: `ST001`
Expected Output	System should not generate a full report and should prompt the user to select a report type
Actual Output	Report area displayed "No report type selected." after the 4-second delay. A popup prompt was not shown
Status	Pass

Execution Log:

Left the report type dropdown empty and clicked Generate Report. The system fell into the else branch of the report logic and displayed the message "No report type selected." in the output area. Although the message appeared, the 4-second delay still occurred even for this empty case

Screenshot Reference:



TASK 4: NON-FUNCTIONAL TESTING**Performance Testing****Description of Method:**

Performance testing measures how quickly the application responds to user actions under normal conditions. The focus of this test is the ****Report Generation**** feature in `FrmReports`, which is the most computationally triggered operation in the application. A stopwatch was used to record elapsed time from the moment the ****Generate Report**** button was clicked to the moment the report text appeared in the output field.

Three report types were tested in sequence:

1. Student Summary Report
2. Marks Report
3. Enrollment Report

Each test was performed immediately after the previous one, with the form cleared between runs using the Clear button.

Test ID	Report Type	Input	Load Time	Threshold
PERF-001	Student Summary Report	STU001	5+sec	< 2sec
PERF-002	Marks Report	STU002	4+sec	< 2sec
PERF-003	Enrollment Report	STU003	5+sec	< 2sec

Acceptable threshold rationale:

A general usability standard for desktop applications expects interactive responses within 2 seconds. Any response exceeding this is considered a performance concern.

Interpretation of Findings

All three report types took approximately 4 seconds to load. This delay is caused by a `Thread.Sleep(4000)` call present inside the `btnGenerateReport_Click` method in `FrmReports.cs`. This call artificially pauses the main UI thread for 4000 milliseconds before any output is produced.

The consequence of this is that the application becomes unresponsive during the 4 second wait; the window cannot be moved, resized, or interacted with. This is a significant performance defect that would be unacceptable in a production system.

Finding: The report loading time exceeds the acceptable threshold by approximately 2 seconds on all report types. The root cause is the `Thread.Sleep(4000)` simulation. In a real system, this would need to be replaced with asynchronous data loading using `async/await` so the UI thread remains responsive during data retrieval.

Defect Reference: BUG-001 Report generation blocks the UI thread for 4 seconds on every execution.

Usability Testing

Description of Method

Usability testing evaluates how easy and intuitive the application is to navigate. The test measured the number of steps (clicks/actions) a user must take to complete each primary task from the login screen. A step is defined as any deliberate user action: a button click, a form field entry, or a dropdown selection.

The tasks tested were:

- Log in to the system
- Register a new student
- Enroll a student in a course
- Capture marks for a student
- Generate a report
- Log out of the system

A tester who had not previously used the application was asked to complete each task while the number of steps was counted. Time taken per task was also noted.

Results:

Test ID	Task	Steps Required	Time Taken	Notes
UI-001	Log in to the system	3 steps (enter username, enter password, click Login)	5+ seconds	Straightforward. No ambiguity
UI-002	Register a new student	7 steps (enter ID, name, email, age, select programme, click Save, click Back)	25+ seconds	Input fields are clearly labelled
UI-003	Enroll a student in a course	6 steps (enter student ID, name, select course, select semester, click Enroll, click Back)	20+ seconds	Dropdowns are pre-populated with options and easy to use
UI-004	Capture student marks	7 steps (enter student ID, name, subject 1, subject 2, subject 3, click Calculate, click Back)	25+ seconds	Labels are clear, but no guidance is provided on valid mark ranges
UI-005	Generate a report	4 steps (select report type, enter student ID, click Generate, click Back)	10+ seconds	The 4-second freeze causes confusion because no loading indicator is displayed
UI-006	Log out of the system	1 step (click Logout on Dashboard)	2+ seconds	Simple and accessible

Navigation path to complete all primary tasks from login:

Login → Dashboard

Dashboard → each module

each module → Back to Dashboard

Back to Dashboard → Logout.

Total steps from cold start to logout: approximately 28–32 steps depending on data entered

Interpretation of Findings

Overall, the application is reasonably usable for a basic desktop system. Navigation is linear and consistent each module opens from the Dashboard and returns via a Back button. Button labels are descriptive and match their actions

Usability concerns identified:

- No loading indicator during report generation When the Generate Report button is clicked, the application freezes for 4 seconds with no visual feedback. A first-time user may believe the system has crashed. A progress bar or status label would significantly improve this experience
- No input guidance on the Marks Capture form There is no indication of the valid mark range 0–100. A user could enter negative values or values above 100 with no warning, as validation is absent

Finding: The application is navigable and logically structured but lacks user feedback during slow operations and does not guide users on valid input ranges in the Marks Capture module

Security Testing**Description of Method**

Security testing evaluates whether the application's access controls are robust enough to prevent unauthorised access. The login form in `FrmLogin.cs` is the sole entry point to the system. The test focused on invalid login attempts using multiple combinations of credentials to verify that the system correctly denies access in all unauthorised scenarios.

The known valid credentials are:

- Username: `admin`
- Password: `1234`

The login condition in the source code uses a logical OR (`||`) operator:

```
if (username = "admin" || password = "1234")
```

This means access is granted if either condition is true, not both. The following test scenarios were designed to probe this vulnerability

Results:

Test ID	Username Entered	Password Entered	Expected Result	Actual Result	Outcome
ST-001	admin	1234	Access granted	Access granted Dashboard opened successfully	Pass
ST-002	admin	wrongpass	Access denied	Access granted Dashboard opened incorrectly	Fail
ST-003	wronguser	1234	Access denied	Access granted Dashboard opened incorrectly	Fail
ST-004	wronguser	wrongpass	Access denied	Access denied Error message displayed	Pass
ST-005	(empty)	1234	Access denied	Access granted Dashboard opened incorrectly	Fail
ST-006	admin	(empty)	Access denied	Access granted Dashboard opened incorrectly	Fail
ST-007	(empty)	(empty)	Access denied	Access denied Error message displayed	Pass

Interpretation of Findings

The login form contains a critical security defect The use of the `||` OR operator instead of `&&` AND means that knowing only the username OR only the password is sufficient to gain full access to the system. This bypasses the fundamental purpose of a two-factor credential check

In a real-world application, this vulnerability would allow any person who knows only the application's username which is often visible or guessable as admin to gain complete access without ever knowing the correct password

All Scenarios represent unauthorised access being incorrectly granted

Defect Reference: BUG-001 previously identified in Task 3 The login condition uses `||` instead of `&&`, granting access when either credential alone is correct

Recommended Fix:

Recommended Fix:

```
// Current (insecure):  
if (username = "admin" || password = "1234")  
  
// Corrected (secure):  
if (username = "admin" && password = "1234")
```

Finding: The login security is critically flawed. The system fails to correctly authenticate users when only one credential is valid, posing a direct and serious access-control vulnerability

Reliability Testing**Description of Method**

Reliability testing evaluates whether the application behaves consistently and correctly when the same operations are performed repeatedly. The goal is to detect instability, crashes, data corruption, or inconsistent output across multiple executions of the same task

Three operations were repeated five times each without restarting the application between runs:

- Student Registration: Save a student record with the same valid input data
- Marks Calculation: Calculate the average and result for the same mark values
- Report Generation : Generate the same report type multiple times

Results:**Operation 1: Student Registration**

Input used each time: `StudentId = STU001`, `FullName = Willem Gerber`, `Email = willem@gmail.com`, `Age = 21`, `Programme = Software Engineering`

Run	Output Consistent	Application Crashed	Notes
1	Yes	No	Output displayed correctly
2	Yes	No	Output identical to Run 1
3	Yes	No	Output identical to Run 1
4	Yes	No	Output identical to Run 1
5	Yes	No	Output identical to Run 1

Operation 2: Marks Calculation

Input used each time: `Subject1 = 60`, `Subject2 = 70`, `Subject3 = 80`

Expected average: `(60 + 70 + 80) / 3 = 70`

Run	Displayed Average	Correct	Application Crashed	Notes
1	156.67	No	No	Operator precedence bug produces wrong result
2	156.67	No	No	Same incorrect result
3	156.67	No	No	Same incorrect result
4	156.67	No	No	Same incorrect result
5	156.67	No	No	Same incorrect result

The faulty calculation in `FrmMarksCapture.cs` is:

FrmMarksCapture.cs

```
// Current incorrect :
record.Average = record.Subject1 + record.Subject2 + record.Subject3 / 3;
// Evaluated as: 60 + 70 + (80 / 3) = 156.67

// Corrected:
record.Average = (record.Subject1 + record.Subject2 + record.Subject3) /
// Evaluated as: (60 + 70 + 80) / 3 = 70
```

Defect Reference:

BUG-MARKS-001 Average calculation uses incorrect operator precedence

Operation 3: Report Generation

Input used each time: `Report Type = Marks Report`, `Student ID = STU001`

Run	Report Output Consistent	Load Time	Application Crashed	Notes
1	Yes	4+ Sec	No	Report generated successfully
2	Yes	4+ Sec	No	Report identical to Run 1
3	Yes	4+ Sec	No	Report identical to Run 1
4	Yes	4+ Sec	No	Report identical to Run 1
5	Yes	4+ Sec	No	Report identical to Run 1

Interpretation of Findings

The application demonstrated stable reliability in the sense that it did not crash during any of the 15 repeated operations. The same inputs consistently produced the same outputs across all runs, which indicates the application has no random failure modes or memory-related instability

However, the reliability tests exposed two consistent defects that affect every run:

- **Marks calculation is always wrong:** The average is calculated incorrectly on every execution due to operator precedence. The result is reliably incorrect, not random meaning the defect is deterministic and will affect every student's result captured through this form
- **Report load time is always slow:** The 4-second delay is present on every report generation run without exception, confirming it is a permanent performance issue, not an intermittent one

Finding: The application is reliable in terms of stability no crashes but consistently reproduces two known defects. Reliability, in this context, means the defects are dependable they will always occur. This is both a reliability concerns the system reliably fails to produce correct marks and a confirmation that the defects are deterministic and must be fixed at the code level

TASK 5

DEBUGGING SOURCE CODE

Bug 1 Incorrect Login Validation

- File: Forms/FrmLogin.cs
- Method: btnLogin_Click
- Category: Incorrect Conditional Statement / Incorrect Login Validation

Original Faulty Code:

Original Faulty Code:

```
private void btnLogin_Click(object sender, EventArgs e)
{
    string username = txtUsername.Text;
    string password = txtPassword.Text;

    if (username = "admin" || password = "1234")
    {
        MessageBox.Show("Login Successful!");

        FrmDashboard dashboard = new FrmDashboard();
        dashboard.Show();
        this.Hide();
    }
    else
    {
        MessageBox.Show("Invalid login credentials.");
    }
}
```

Identified Issue:

The conditional operator on line `if (username == "admin" || password == "1234")` uses the **logical OR (`||`)** operator instead of the **logical AND (`&&`)** operator.

With `||`, the condition evaluates to `true` if **either** sub-condition is true. This means:

- A user who enters only the correct username (`admin`) with any wrong password gains full access.
- A user who enters only the correct password (`1234`) with any wrong username gains full access.
- A user who enters both fields empty still has both conditions evaluated as false and is denied but this is the only scenario where the gate holds.

A login check must require both credentials to be correct simultaneously. Only the `&&` operator enforces this requirement.

Impact: This is a critical security vulnerability. Any person who knows either the username alone or the password alone can bypass authentication and access the entire application

Corrected Version:

Corrected Version

```
private void btnLogin_Click(object sender, EventArgs e)
{
    string username = txtUsername.Text;
    string password = txtPassword.Text;

    if (username == "admin" && password == "1234")
    {
        MessageBox.Show("Login Successful!");

        FrmDashboard dashboard = new FrmDashboard();
        dashboard.Show();
        this.Hide();
    }
    else
    {
        MessageBox.Show("Invalid login credentials.");
    }
}
```

Explanation of Fix:

The single character change from `||` to `&&` corrects the authentication logic. With `&&`, the condition only evaluates to `true` when **both** `username == "admin"` AND `password == "1234"` are simultaneously true. Any other combination — wrong username, wrong password, or either field empty — will now correctly fall through to the `else` branch and display the invalid credentials message. Access is only granted when the full credential pair is matched

Bug 2 Incorrect Average Calculation:

- File: Forms/FrmMarksCapture.cs
- Method: btnCalculateResults_Click
- Category: Incorrect Average Calculation

Original Faulty Code:

```
Original Faulty Code:

record.Subject1 = Convert.ToDouble(txtSubject1.Text);
record.Subject2 = Convert.ToDouble(txtSubject2.Text);
record.Subject3 = Convert.ToDouble(txtSubject3.Text);

record.Average = record.Subject1 + record.Subject2 + record.Subject3 / 3;

if (record.Average ≥ 50)
{
    record.ResultStatus = "PASS";
}
else
{
    record.ResultStatus = "FAIL";
}
```

Identified Issue:

The average calculation on line `record.Average = record.Subject1 + record.Subject2 + record.Subject3 / 3;` contains an **operator precedence error**.

In C#, the division operator `/` has higher precedence than the addition operator `+`. This means the expression is evaluated as:

Average = Subject1 + Subject2 + (Subject3 / 3)

****Not**** as the intended:

Average = (Subject1 + Subject2 + Subject3) / 3

Concrete example:

using Subject1 = 60, Subject2 = 70, Subject3 = 80:

Type:	Calculation	Result
Faulty:	$60 + 70 + (80 / 3)$	$60 + 70 + 26.67 = 156.67$
Correct:	$(60 + 70 + 80) / 3$	$210 / 3 = 70.00$

The faulty result of 156.67 also corrupts the pass/fail determination. Because $156.67 \geq 50$ is always true for any positive marks, every student will receive a Pass status regardless of their actual performance.

Impact: Every student's academic record is computed incorrectly. A student scoring 10, 10, 10 would receive an average of 23.33 faulty and a Pass result, when their true average is 10.00 and the result should be Fail

Corrected Version:

Corrected Version

```
record.Subject1 = Convert.ToDouble(txtSubject1.Text);
record.Subject2 = Convert.ToDouble(txtSubject2.Text);
record.Subject3 = Convert.ToDouble(txtSubject3.Text);

record.Average = (record.Subject1 + record.Subject2 + record.Subject3) / 3;

if (record.Average ≥ 50)
{
    record.ResultStatus = "PASS";
}
else
{
    record.ResultStatus = "FAIL";
}
```

Explanation of Fix:

Wrapping the addition in parentheses `(record.Subject1 + record.Subject2 + record.Subject3)` forces the three marks to be summed first before the division by 3 is applied. This correctly follows the mathematical formula for an arithmetic mean:

arithmetic mean

$$\text{Average} = \frac{\text{Subject1} + \text{Subject2} + \text{Subject3}}{3}$$

The parentheses override C#'s default operator precedence rules, ensuring the division applies to the total sum rather than to Subject3 alone. The pass/fail conditional `(>= 50)` remains correct and now operates on a valid average value

Bug 3 Missing Exception Handling in Student Registration

- File: Forms/FrmStudentRegistration.cs
- Method: btnSaveStudent_Click
- Category: Missing Exception Handling

Original Faulty Code:

Original Faulty Code

```
private void btnSaveStudent_Click(object sender, EventArgs e)
{
    Student student = new Student
    {
        StudentId = txtStudentId.Text,
        FullName = txtFullName.Text,
        Email = txtEmail.Text,
        Age = int.Parse(txtAge.Text),
        Programme = cmbProgramme.Text
    };

    txtStudentOutput.Text =
        "Student saved successfully!" + Environment.NewLine +
        "Student ID: " + student.StudentId + Environment.NewLine +
        "Full Name: " + student.FullName + Environment.NewLine +
        "Email: " + student.Email + Environment.NewLine +
        "Age: " + student.Age + Environment.NewLine +
        "Programme: " + student.Programme;
}
```

Identified Issue:

The line `Age = int.Parse(txtAge.Text)` uses `int.Parse()` to convert the text field value directly to an integer. `int.Parse()` throws an unhandled `FormatException` if the input string cannot be converted to an integer.

This means:

- If the user types letters the application crashes with an unhandled exception.
- If the user types a decimal the application crashes.
- If the Age field is left empty, the application crashes.
- If the user types a negative number or an impossibly large age no warning is given and invalid data is saved.

Because there is no `try-catch` block and no input validation before `int.Parse` is called, any non-integer input terminates the method with an unhandled exception, which in a Windows Forms application produces an error dialog and may leave the form in an inconsistent state.

Corrected Version:

Corrected Version

```
private void btnSaveStudent_Click(object sender, EventArgs e)
{
    if (!int.TryParse(txtAge.Text, out int age) || age ≤ 0 || age > 120)
    {
        MessageBox.Show("Please enter a valid age (a whole number between 1
and 120).",
                        "Invalid Input", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        txtAge.Focus();
        return;
    }

    Student student = new Student
    {
        StudentId = txtStudentId.Text,
        FullName = txtFullName.Text,
        Email = txtEmail.Text,
        Age = age,
        Programme = cmbProgramme.Text
    };

    txtStudentOutput.Text =
        "Student saved successfully!" + Environment.NewLine +
        "Student ID: " + student.StudentId + Environment.NewLine +
        "Full Name: " + student.FullName + Environment.NewLine +
        "Email: " + student.Email + Environment.NewLine +
        "Age: " + student.Age + Environment.NewLine +
        "Programme: " + student.Programme;
}
```

Explanation of Fix:

int.Parse() is replaced with int.TryParse(). Unlike int.Parse(), int.TryParse() does not throw an exception on invalid input. Instead, it returns false if the conversion fails, allowing the code to handle the invalid case

The condition int.TryParse(txtAge.Text, out int age) evaluates to true (meaning: parsing failed) whenever the input is not a valid integer. The additional checks age ≤ 0 and age > 120 reject values that are technically parseable integers but are not realistic ages

When invalid input is detected, a descriptive warning message is shown to the user, the Age field is focused so the user can correct it immediately, and the return statement exits the method early without attempting to create a Student object with bad data. No crash occurs

Bug 4 Missing Exception Handling in Marks Capture:

- **File:** Forms/FrmMarksCapture.cs
- **Method:** btnCalculateResults_Click
- **Category:** Missing Exception Handling

Original Faulty Code:

Original Faulty Code

```
private void btnCalculateResults_Click(object sender, EventArgs e)
{
    MarkRecord record = new MarkRecord();

    record.StudentId = txtMarkStudentId.Text;
    record.StudentName = txtMarkStudentName.Text;
    record.Subject1 = Convert.ToDouble(txtSubject1.Text);
    record.Subject2 = Convert.ToDouble(txtSubject2.Text);
    record.Subject3 = Convert.ToDouble(txtSubject3.Text);

    record.Average = record.Subject1 + record.Subject2 + record.Subject3 /
3;

    if (record.Average ≥ 50)
    {
        record.ResultStatus = "PASS";
    }
    else
    {
        record.ResultStatus = "FAIL";
    }

    txtMarksOutput.Text =
        "Marks processed successfully!" + Environment.NewLine +
        "Student ID: " + record.StudentId + Environment.NewLine +
        "Student Name: " + record.StudentName + Environment.NewLine +
        "Subject 1: " + record.Subject1 + Environment.NewLine +
        "Subject 2: " + record.Subject2 + Environment.NewLine +
        "Subject 3: " + record.Subject3 + Environment.NewLine +
        "Average: " + record.Average + Environment.NewLine +
        "Final Result: " + record.ResultStatus;
}
```

Identified Issue:**The three lines:**

```
record.Subject1 = Convert.ToDouble(txtSubject1.Text);
record.Subject2 = Convert.ToDouble(txtSubject2.Text);
record.Subject3 = Convert.ToDouble(txtSubject3.Text);
```

use Convert.ToDouble() to parse text field values directly. Convert.ToDouble() throws a FormatException when the input string is not a valid numeric value, and an OverflowException when the value is outside the range of a double

This means:

- If any subject field is left empty, the application crashes.
- If a user types letters the application crashes.
- If a user types a value malformed decimal the application crashes.
- There is also no check that mark values fall within a valid academic range

No `try-catch` block exists to intercept these exceptions, so an unhandled exception is thrown and the application fails without providing any useful guidance to the user

Corrected Version:

Corrected Version:

```

private void btnCalculateResults_Click(object sender, EventArgs e)
{
    if (!double.TryParse(txtSubject1.Text, out double subject1) ||
        !double.TryParse(txtSubject2.Text, out double subject2) ||
        !double.TryParse(txtSubject3.Text, out double subject3))
    {
        MessageBox.Show("Please enter valid numeric mark values for all
three subjects.",
                        "Invalid Input", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        return;
    }

    if (subject1 < 0 || subject1 > 100 ||
        subject2 < 0 || subject2 > 100 ||
        subject3 < 0 || subject3 > 100)
    {
        MessageBox.Show("Mark values must be between 0 and 100.",
                        "Invalid Input", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        return;
    }

    MarkRecord record = new MarkRecord();

    record.StudentId = txtMarkStudentId.Text;
    record.StudentName = txtMarkStudentName.Text;
    record.Subject1 = subject1;
    record.Subject2 = subject2;
    record.Subject3 = subject3;

    record.Average = (record.Subject1 + record.Subject2 + record.Subject3)
/ 3;

    if (record.Average ≥ 50)
    {
        record.ResultStatus = "PASS";
    }
    else
    {
        record.ResultStatus = "FAIL";
    }

    txtMarksOutput.Text =
        "Marks processed successfully!" + Environment.NewLine +
        "Student ID: " + record.StudentId + Environment.NewLine +
        "Student Name: " + record.StudentName + Environment.NewLine +
        "Subject 1: " + record.Subject1 + Environment.NewLine +
        "Subject 2: " + record.Subject2 + Environment.NewLine +
        "Subject 3: " + record.Subject3 + Environment.NewLine +
        "Average: " + record.Average + Environment.NewLine +
        "Final Result: " + record.ResultStatus;
}

```

Explanation of Fix:

Convert.ToDouble() is replaced with double.TryParse() for all three subject fields. All three conversions are checked in a single if statement using || if any of the three fields fails to parse as a valid double, the method shows a warning and exits early via return. No exception is thrown. A second validation block checks that each parsed value falls within the valid academic range of 0 to 100. This catches inputs that are technically numeric but are not valid marks.

TASK 6
DEFECT REPORTING:

Bug ID	Module	Description	Severity	Priority	Status	Resolution
BUG-001	Login (FrmLogin)	Login validation uses OR instead of AND — a user can log in with only the correct username OR only the correct password. For example, entering any username with password 1234 grants access, bypassing authentication entirely	Critical	High	Open	Change the condition from <code>username == "admin" password == "1234"</code> to <code>username == "admin" && password == "1234"</code> so that both credentials are required
BUG-002	Marks Capture (FrmMarksCapture)	Average calculation is incorrect due to operator precedence. The formula <code>Subject1 + Subject2 + Subject3 / 3</code> only divides Subject 3 by 3 before adding the others, producing an incorrect result. For example, marks 60, 70, and 90 produce 160 instead of the correct average of 73.33	High	High	Open	Wrap the total in parentheses: <code>(Subject1 + Subject2 + Subject3) / 3</code> to ensure the entire sum is divided correctly
BUG-003	Student Registration (FrmStudentRegistration)	No input validation exists for the Age field. Calling <code>int.Parse(txtAge.Text)</code> throws an unhandled <code>FormatException</code> if the field is empty or contains non-numeric characters, causing the application to crash	High	High	Open	Replace <code>int.Parse()</code> with <code>int.TryParse()</code> and validate the input before processing. Display a user-friendly error message if the value is invalid
BUG-004	Reports (FrmReports)	Report generation calls <code>Thread.Sleep(4000)</code> on the UI thread, causing the application to freeze and become unresponsive for 4 seconds whenever a report is generated.	Medium	Medium	Open	Move report generation to a background thread using <code>Task.Run()</code> or <code>BackgroundWorker</code> , then update the UI using <code>Invoke()</code> after completion
BUG-005	Reports (FrmReports)	The hardcoded Marks Report displays an incorrect average of 169 instead of the correct value of 74.33 for marks 78, 65, and 80. This provides false academic information to users	Medium	Medium	Open	Correct the hardcoded average to 74.33 or calculate the average dynamically from the displayed marks
BUG-006	Course Enrollment (FrmCourseEnrollment)	No validation is performed before saving an enrollment. Clicking the Enroll button with empty fields saves a blank enrollment record and still displays a success message	Medium	High	Open	

TASK 7 SOFTWARE TESTING REPORT:**Executive Summary**

This report presents the findings of a structured quality assurance (QA) testing engagement conducted on the ULMS Windows Forms desktop application. The objective of the testing process was to evaluate the functional correctness, input validation robustness, security integrity, and performance behaviour of the system across all implemented modules

Testing was conducted through manual black-box testing, white-box source code review, and static code analysis. A total of 7 defects were identified across 5 modules. Of these, 1 defect is rated Critical 2 are rated High 3 are rated Medium and 1 is rated Low

The most significant finding is a critical authentication bypass vulnerability in the Login module, which allows unauthorised access using only the correct password without a valid username. This defect alone is sufficient to block deployment until resolved

Additional defects include a faulty average calculation in the Marks Capture module, unhandled exceptions caused by missing input validation, a UI-freezing performance issue in the Reports module, and missing data integrity checks across multiple forms

Deployment Readiness: Not Ready Critical and High severity defects must be resolved before the system can be approved for production release

Testing Scope

Module	Form File	In Scope
Login & Authentication	Forms/FrmLogin.cs	Yes
Student Registration	Forms/FrmStudentRegistration.cs	Yes
Course Enrollment	Forms/FrmCourseEnrollment.cs	Yes
Marks Capture	Forms/FrmMarksCapture.cs	Yes
Reports	Forms/FrmReports.cs	Yes
Dashboard Navigation	Forms/FrmDashboard.cs	Yes Test only

Testing Types Performed

- Functional Testing: Verifying each feature behaves according to its expected outcome
- Boundary Value Testing: Testing input fields with empty, minimum, maximum, and invalid values
- Negative Testing: Entering invalid, unexpected, or malicious input to observe application behaviour
- White-Box / Code Review: Reviewing source code directly for logic errors, missing validation, and security weaknesses
- -Static Analysis: Manually inspecting code structure for operator errors, exception handling gaps, and hardcoded values
- Performance Testing (Informal): Observing UI responsiveness during report generation

Out of Scope

- Database integration testing no database connection is implemented in this version
- Network/API testing
- Automated unit testing
- Load and stress testing

Tools Used:

Tool	Purpose
Visual Studio 2022	IDE used to run, debug, and inspect the application source code
.NET 8 SDK	Build and runtime environment for the application
Windows 11	Host operating system used for manual testing
Markdown	Used to document test plans, case designs, and this report

Defects Identified:

Bug ID	Module	Description	Severity	Priority	Status
BUG-001	Login	OR logic in login condition allows authentication bypass	Critical	High	Open
BUG-002	Marks Capture	Incorrect operator precedence produces wrong average mark	High	High	Open
BUG-003	Student Registration	int.Parse() crashes app on empty or non numeric Age input	High	High	Open
BUG-004	Reports	Thread.Sleep(4000) on UI thread freezes the application	Medium	Medium	Open
BUG-005	Reports	Hardcoded average of 169 is mathematically incorrect	Medium	Medium	Open
BUG-006	Course Enrollment	No field validation blank enrollment saved as successful	Medium	High	Open
BUG-007	Student Registration	No email format validation allows corrupt data to be saved	Low	Medium	Open

Corrections Implemented:

BUG-001 Login Authentication Bypass

Correction: Change the logical operator in FrmLogin.cs from || to && this ensures both the username and password must be correct simultaneously before granting access

```
if (username = "admin" || password = "1234")
```

```
if (username = "admin" && password = "1234")
```

BUG-002 Incorrect Average Calculation

Correction: Wrap the addition in parentheses in FrmMarksCapture.cs to enforce correct operator precedence

```
// Faulty
record.Average = record.Subject1 + record.Subject2 + record.Subject3 / 3;

// Corrected
record.Average = (record.Subject1 + record.Subject2 + record.Subject3) / 3;
```

BUG-003 Unhandled Exception on Age Field

Correction: Replace `int.Parse()` with `int.TryParse()` and add validation in `FrmStudentRegistration.cs`

```
// Faulty
Age = int.Parse(txtAge.Text),

// Corrected
if (!int.TryParse(txtAge.Text, out int age) || age ≤ 0)
{
    MessageBox.Show("Please enter a valid age.");
    return;
}
```

BUG-004 UI Thread Freezing

Correction: Replace `Thread.Sleep(4000)` with an asynchronous approach in `FrmReports.cs` using `Task.Run()` to process on a background thread and update the UI via `Invoke()`, preventing the window from becoming unresponsive

BUG-005 Hardcoded Incorrect Average in Report

Correction: Update the hardcoded value Average: 169 to the correct calculated value Average: 74.33 in the Marks Report section of `FrmReports.cs`, or replace hardcoded output with a dynamic calculation

BUG-006 Missing Enrollment Validation

Correction: Add guard clauses in `FrmCourseEnrollment.cs` to check that Student ID, Student Name, Course, and Semester fields are not empty before allowing enrollment to proceed

BUG-007 Missing Email Format Validation

Correction: Add email format validation in `FrmStudentRegistration.cs` using a regular expression or `System.Net.Mail.MailAddress` to reject invalid email strings before saving the record

Recommendations:

Based on the findings of this testing engagement, the following recommendations are made

1. **Fix BUG-001 immediately** The authentication bypass is a critical security vulnerability. Deploying the system in its current state would allow any user who knows only the password to gain administrator access. This must be the highest priority fix.
2. **Implement centralised input validation** Multiple forms (Student Registration, Course Enrollment) accept and process empty or invalid data. A shared validation helper class should be introduced to enforce consistent rules across all forms.
3. **Replace all** `int.Parse()` calls with `int.TryParse()`. Any direct `.Parse()` call without a try-catch or `TryParse` pattern is a crash risk. All numeric input fields across the application should be reviewed and hardened.
4. **Adopt async/await for long-running operations** The use of `Thread.Sleep()` on the UI thread in `FrmReports.cs` is a poor practice. All I/O-bound or time-consuming operations should be moved off the UI thread using `async/await` or `BackgroundWorker`.
5. **Remove or replace all hardcoded data** The Reports module uses hardcoded student names, marks, and calculated values. In a production system, this data must be retrieved dynamically from a data source.
6. **Introduce automated unit testing** The intentional defects in this codebase demonstrate the value of automated tests. A test project using a framework such as `MSTest` or `JUnit` should be added to verify the correctness of business logic (average calculation) independently of the UI.
7. **Conduct a security review before deployment** Beyond BUG-001, the application should be reviewed for other security concerns including hardcoded credentials, lack of password hashing, and absence of session management.

Deployment Readiness Decision

My Opinion: Not Ready for Deployment

Criteria	Result
All Critical defects resolved	No BUG-001 is unresolved
All High defects resolved	No BUG-002 and BUG-003 are unresolved
Input validation sufficient	No Multiple forms lack basic validation
UI performance acceptable	No UI freezes during report generation
Data integrity assured	No Invalid/blank data can be saved

The application must not be deployed to production in its current state. A minimum of BUG-001, BUG-002, and BUG-003 must be fully corrected and re-tested before a re-assessment for deployment readiness can be conducted.

Following resolution of all Critical and High defects, a regression test cycle should be performed to confirm that no new defects have been introduced.

Once all identified defects are resolved and verified, the application may be resubmitted for a final QA sign-off and deployment approval.

GitHub Link:

<https://github.com/Gerber-W/ULMSWinFormsAppClient>

Faculty of Information Technology									
I declare that I am familiar with, and will abide to the Examination rules of CTU  Signature	SUBJECT NAME: Solution Development SUBJECT CODE: SDT621								
	Formative Assessment 3 Duration: Date: Total Marks: Total pages:					Examiner: Moderator:			
	Student number:								
	2	0	2	3	0	2	5	7	
	Surname: Gerber					Initials: W		/	

2026/05/08